

DEVOPS TOOLS E CLOUD COMPUTING

VESTA

GABRIEL CRUZ FERREIRA | RM 559613

JOÃO VICTOR IGNACIO MADELLA | RM 561007

KAUÃ FERREIRA DOS SANTOS | RM 560992

NATHÁLIA MANTOVANI DE FALCO | RM 99904

VINICIUS DA SILVA BITÚ | RM 560227

FIAP

SUMÁRIO

- 1 .Introdução
2. Objetivo
3. Descrição da Solução
4. Arquitetura da Solução
5. Tecnologias Utilizadas
6. Azure Boards
- 7.Azure Repos
- 8.Azure Pipelines
9. Provisionamento da Infraestrutura
10. Banco de Dados
11. Relacionamentos Utilizados
12. Operações CRUD
13. Evidências da Implementação
14. Segurança
- 15 .Resultados Obtidos
- 16 .Conclusão
- 17 . Links da Entrega

1. INTRODUÇÃO

O projeto Vesta foi desenvolvido com a finalidade de apoiar instituições de acolhimento no gerenciamento de usuários, famílias, abrigos e recursos disponíveis.

A aplicação foi construída utilizando Java com Spring Boot, adotando uma arquitetura baseada em APIs REST e banco de dados Oracle para armazenamento das informações.

Além do desenvolvimento da aplicação, foi implementada uma esteira DevOps utilizando Azure DevOps, permitindo automação dos processos de integração contínua e entrega contínua.

2. OBJETIVO

O objetivo deste trabalho consiste em aplicar os conceitos de DevOps por meio da implantação da solução Vesta em ambiente de nuvem.

A implementação contempla:

- Controle de versão utilizando Git;
- Gerenciamento de atividades com Azure Boards;
- Integração do código através do Azure Repos;
- Automação de Build e Deploy através do Azure Pipelines;
- Provisionamento da infraestrutura utilizando Azure CLI;
- Hospedagem da aplicação em ambiente Azure;
- Persistência dos dados em Oracle Database.

3. DESCRIÇÃO DA SOLUÇÃO

O Vesta é uma aplicação voltada ao gerenciamento de instituições de acolhimento.

A solução permite:

- Cadastro de usuários;
- Cadastro de abrigos;
- Cadastro de famílias;
- Cadastro de pessoas acolhidas;
- Controle de recursos;
- Controle de estoque;
- Registro de ocorrências;
- Solicitações de recursos;
- Controle de transferências entre abrigos;
- Geração de alertas.

Todos os dados são disponibilizados através de endpoints REST e armazenados em banco Oracle.

4. ARQUITETURA DA SOLUÇÃO

A arquitetura foi estruturada para permitir o fluxo completo entre desenvolvimento, integração e publicação da aplicação.

Fluxo da solução:

Azure Boards → Azure Repos → Azure Pipelines → Azure App Service → Oracle Database

Fluxo operacional:

1. Criação da atividade no Azure Boards.
2. Desenvolvimento da funcionalidade.
3. Commit do código no Azure Repos.
4. Execução automática da Pipeline.
5. Geração dos artefatos.
6. Deploy da aplicação.
7. Disponibilização da API.
8. Persistência dos dados em Oracle.

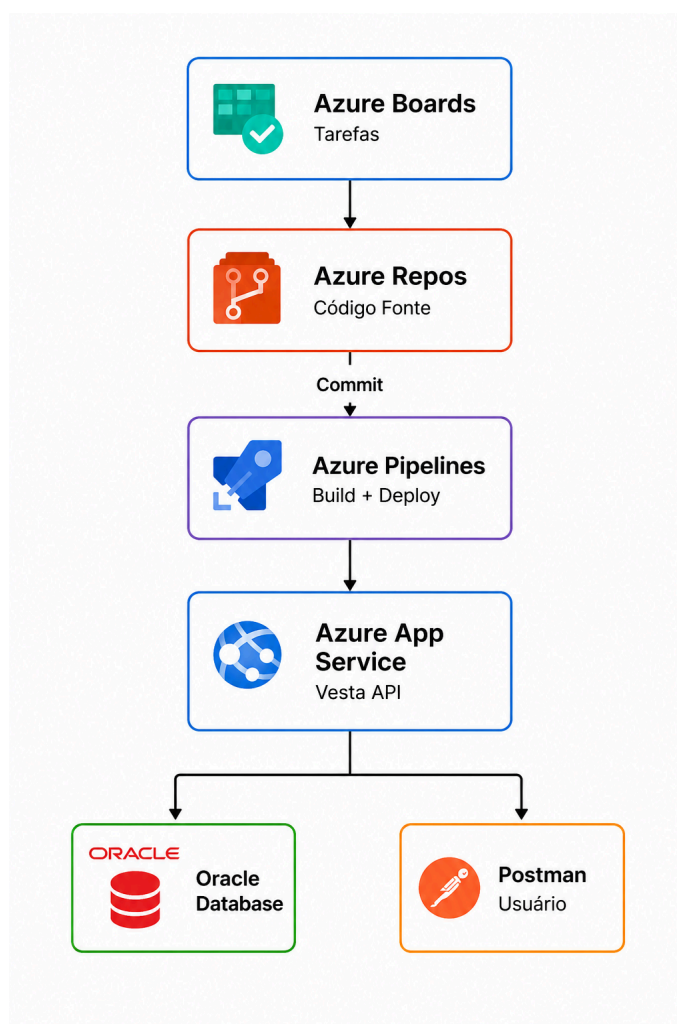


Figura 1 – Arquitetura da Solução Vesta.

5. TECNOLOGIAS UTILIZADAS

Backend

- Java 21
- Spring Boot
- Maven

Banco de Dados

- Oracle Database

DevOps

- Azure DevOps
- Azure Boards
- Azure Repos
- Azure Pipelines
- Azure CLI

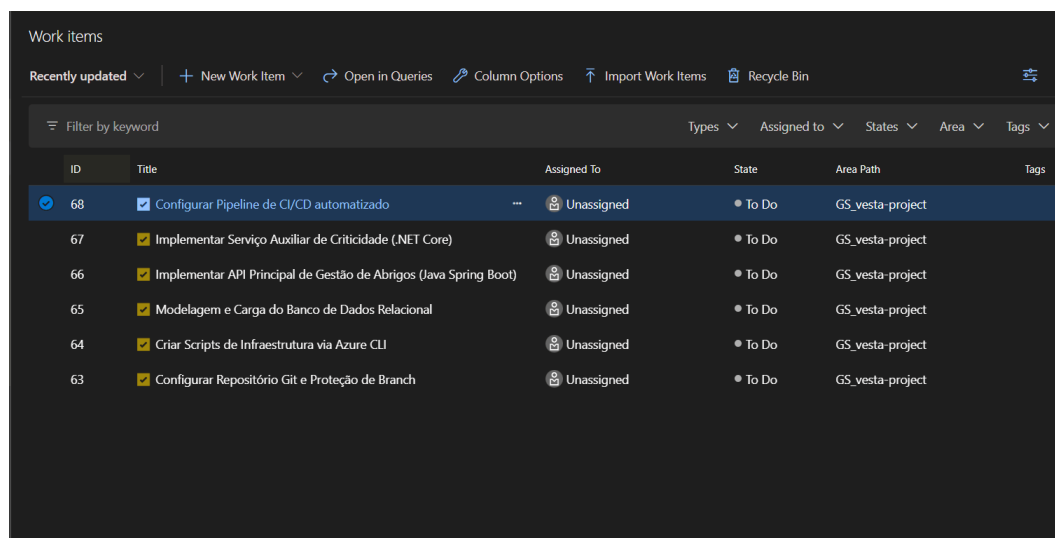
Ferramentas de Teste

- Swagger
- Postman

6. AZURE BOARDS

Foi criada uma atividade para acompanhamento da implementação da esteira DevOps.

A atividade foi utilizada para registrar as etapas de desenvolvimento da solução e permitiu a vinculação entre commits, branches, pull requests e pipelines.

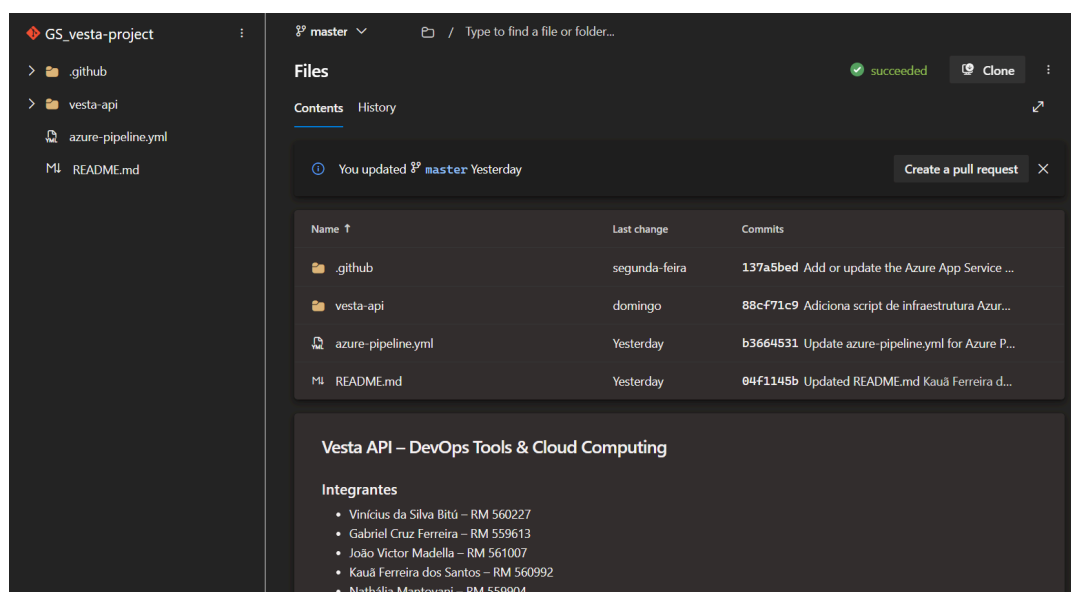


ID	Title	Assigned To	State	Area Path	Tags
68	Configurar Pipeline de CI/CD automatizado	Unassigned	To Do	GS_vesta-project	
67	Implementar Serviço Auxiliar de Criticidade (.NET Core)	Unassigned	To Do	GS_vesta-project	
66	Implementar API Principal de Gestão de Abrigos (Java Spring Boot)	Unassigned	To Do	GS_vesta-project	
65	Modelagem e Carga do Banco de Dados Relacional	Unassigned	To Do	GS_vesta-project	
64	Criar Scripts de Infraestrutura via Azure CLI	Unassigned	To Do	GS_vesta-project	
63	Configurar Repositório Git e Proteção de Branch	Unassigned	To Do	GS_vesta-project	

Figura 2 – Tarefa criada no Azure Boards.

7. AZURE REPOS

O código-fonte da aplicação foi armazenado em repositório Git privado. Foi utilizada uma estratégia baseada em branches para desenvolvimento das funcionalidades e integração posterior à branch principal.



Name ↑	Last change	Commits
.github	segunda-feira	137a5bed Add or update the Azure App Service ...
vesta-api	domingo	88cf71c9 Adiciona script de infraestrutura Azur...
azure-pipeline.yml	Yesterday	b3664531 Update azure-pipeline.yml for Azure P...
README.md	Yesterday	04f1145b Updated README.md Kauã Ferreira d...

Figura 3 – Repositório da aplicação Vesta.

8. AZURE PIPELINES

A automação foi implementada através de pipeline configurada em YAML.

As principais etapas executadas são:

- Checkout do código;
- Download das dependências Maven;
- Compilação da aplicação;
- Empacotamento do projeto;
- Publicação do artefato;
- Deploy automático da aplicação.

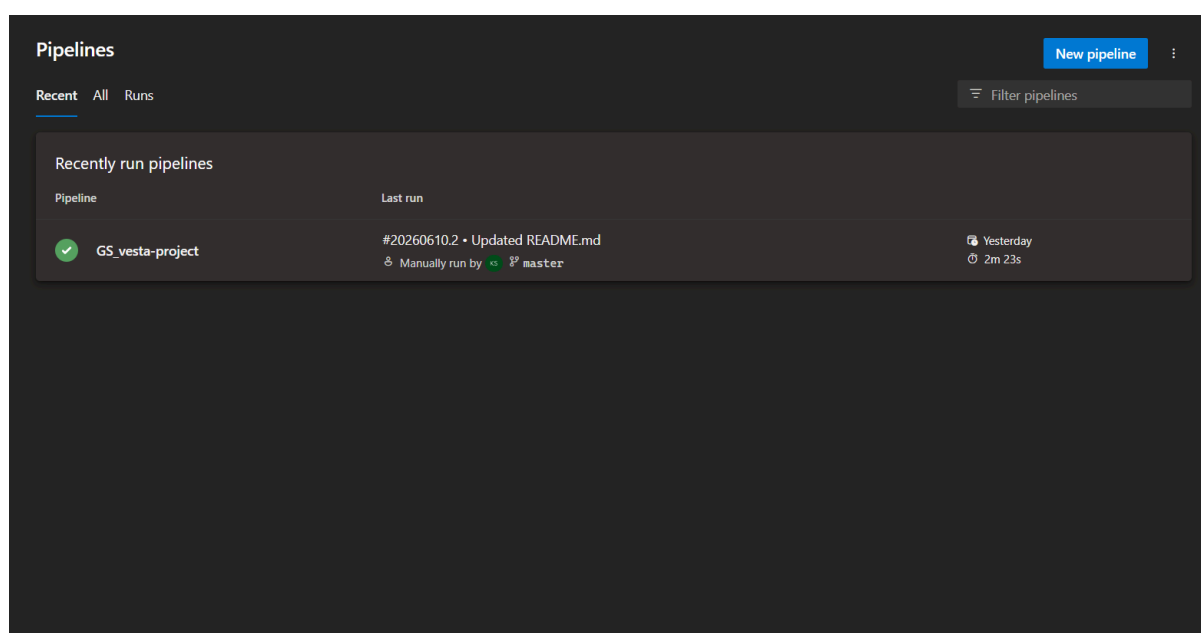


Figura 4 – Pipeline configurada no Azure DevOps.

Description	Stages	Run Time
#20260610.2 - Updated README.md Manually run by master 04f1145b	✓-✓	Yesterday 2m 23s
#20260610.1 - Updated README.md Manually run by master 04f1145b	✓-✓	Yesterday 3m 52s
#20260609.9 - Updated README.md Manually run by master 04f1145b	✓-✓	Yesterday 2m 39s
#20260609.8 - Updated README.md Individual CI for master 04f1145b	✓-✓	Yesterday 2m 40s
#20260609.7 - Update azure-pipeline.yml for Azure Pipelines Manually run by master b3664531	✓-✓	Yesterday 5m 22s
#20260609.6 - Update azure-pipeline.yml for Azure Pipelines Individual CI for master b3664531	✓-✓	Yesterday 4m 50s
#20260609.5 - Update azure-pipeline.yml for Azure Pipelines Manually run by master c7f4ade9	✓-✓	Yesterday 2m 48s
#20260609.4 - Update azure-pipeline.yml for Azure Pipelines Manually run by master c7f4ade9	✓-✓	Yesterday 2m 57s

Figura 5 – Execuções da Pipeline.

master

GS_vesta-project / azure-pipeline.yml

```

1 trigger:
2   - master
3
4 pool:
5   - vmImage: 'ubuntu-22.04'
6
7 variables:
8   - MAVEN_OPTS: '-Dmaven.repo.local=$(Pipeline.Workspace)/.m2/repository'
9
10 stages:
11   - stage: Build
12     displayName: Build Java
13     jobs:
14       - job: BuildJava
15         steps:
16           - task: Cache@2
17             inputs:
18               key: 'maven | "$(Agent.OS)" | vesta-api/pom.xml'
19               restoreKeys: |
20                 - maven | "$(Agent.OS)"
21               path: $(Pipeline.Workspace)/.m2/repository
22
23           - task: JavaToolInstaller@0
24             inputs:
25               versionSpec: '17'
26               jdkArchitectureOption: 'x64'
27               jdkSourceOption: 'PreInstalled'

```

Tasks

Search tasks

- .NET Core: Build, test, package, or publish a dotnet application...
- Android signing: Sign and align Android APK files
- Ant: Build with Apache Ant
- App Center distribute: Distribute app builds to testers and users via Visu...
- App Center test: Test app packages with Visual Studio App Center
- Archive files: Compress files into .7z, .tar.gz, or .zip
- ARM template deployment: Deploy an Azure Resource Manager (ARM) templ...
- Azure App Configuration Export: Export key-values from Azure App Configuration L...

Figura 6 – Arquivo azure-pipelines.yml.

9. PROVISIONAMENTO DA INFRAESTRUTURA

Todos os recursos necessários para execução da aplicação foram criados utilizando Azure CLI.

Recursos provisionados:

- Resource Group: rg-vesta
- App Service Plan: plan-vesta
- Azure Web App: vestags

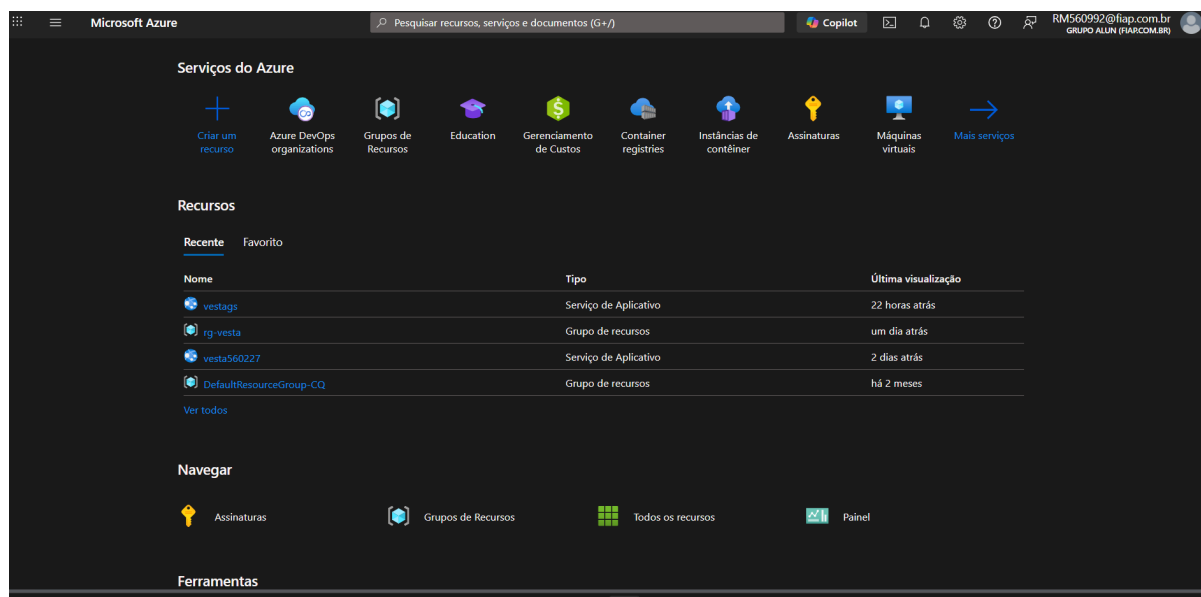


Figura 7 – Recursos provisionados na Azure.

10. BANCO DE DADOS

A aplicação utiliza Oracle Database para armazenamento das informações.

Principais tabelas:

- TB_REGIAO
- TB_INSTITUICAO
- TB_ABRIGO

- TB_USUARIO
- TB_FAMILIA
- TB_PESSOA_ABRIGADA
- TB_RECURSO
- TB_ESTOQUE_ABRIGO
- TB_OCORRENCIA
- TB_SOLICITACAO_RECURSO
- TB_MOVIMENTACAO_RECURSO
- TB_TRANSFERENCIA_ABRIGO
- TB_ALERTA

11. RELACIONAMENTOS UTILIZADOS

Relacionamento demonstrado durante os testes:

TB_ABRIGO → TB_ESTOQUE_ABRIGO → TB_RECURSO

Esse relacionamento permite controlar os recursos existentes em cada abrigo e a quantidade disponível em estoque.

12. OPERAÇÕES CRUD

Durante a validação da aplicação foram executadas operações completas de CRUD.

Create – Cadastro de novos registros.

Read – Consulta dos registros cadastrados.

Update – Atualização das informações existentes.

Delete – Remoção dos registros cadastrados.

Todas as operações foram validadas diretamente no banco Oracle.

13. EVIDÊNCIAS DA IMPLEMENTAÇÃO

Persistência de Dados

Consulta realizada:

`SELECT * FROM TB_ABRIGO;`

Resultado:

12 registros encontrados.



ID_ABRIGO	NM_ABRIGO	DS_ENDEREÇO	QT_CAPACIDADE_MAXIMA	QT_OCUPACAO_ATUAL	ST_STATUS	ID_REGIAO	ID_INSTITUICAO	DT_ABERTURA	DT_CRIACAO
1	1Abrigo Ibirapuera	Av. Pedro Álvares Cabral, 1301 - SP	300	247 ATIVO	1	1	1	07/06/26	07/06/26
2	2Abrigo Ginásio Norte	R. das Palmeiras, 50 - SP	200	200 LOTADO	1	1	1	07/06/26	07/06/26
3	3Abrigo Escola Municipal	Rua Tiradentes, 22 - SJC	150	98 ATIVO	2	2	2	07/06/26	07/06/26
4	4Abrigo Estadual Leste	Av. Brasil, 5000 - Duque de Caxias	400	312 ATIVO	3	3	3	07/06/26	07/06/26
5	5Abrigo Estádio Municipal	Rua do Sport, 10 - Nova Iguaçu	350	0 INATIVO	3	3	3	07/06/26	07/06/26
6	6Abrigo SESC Contagem	Av. João César, 234 - Contagem	250	189 ATIVO	4	4	4	07/06/26	07/06/26
7	7Abrigo Centro BH	Praça Sete, 100 - BH	180	55 ATIVO	4	4	4	07/06/26	07/06/26
8	8Abrigo Pelotas Sul	R. XV de Novembro, 300 - Pelotas	220	198 ATIVO	5	5	5	07/06/26	07/06/26
9	9Abrigo Porto Alegre	Av. Borges de Medeiros, 1700 - POA	500	421 ATIVO	5	5	5	07/06/26	07/06/26
10	10Abrigo Escola RS	Rua Uruguai, 45 - Rio Grande	100	78 ATIVO	5	5	5	07/06/26	07/06/26
11	11Ginásio Poliesportivo Central	Rua das Palmeiras, 450 - Centro	150	142 ATIVO	1	1	1	07/06/26	07/06/26
12	12Abrigo Centro Comunitário	Av. Brasil, 750 - Centro	120	108 ATIVO	1	1	1	09/06/26	09/06/26

Figura 8 – Consulta da tabela TB_ABRIGO.

Consulta realizada:

`SELECT * FROM TB_ESTOQUE_ABRIGO;`

Resultado:

36 registros encontrados.

2
3

```
select * from tb_estoque_abrigo;
```

Resultado da Consulta x Resultado da Consulta 1 x

Todas as Linhas Extraídas: 36 em 0,009 segundos

ID_ESTOQUE	ID_ABRIGO	ID_RECURSO	QT_ATUAL	QT_MINIMA	DT_ATUALIZACAO
9	9	3	3	100	60 07/06/26
10	10	3	9	120	100 07/06/26
11	11	4	1	1200	1600 07/06/26
12	12	4	2	200	160 07/06/26
13	13	4	7	30	80 07/06/26
14	14	4	6	25	40 07/06/26
15	15	6	1	750	500 07/06/26
16	16	6	4	200	125 07/06/26
17	17	6	8	40	60 07/06/26
18	18	8	1	300	400 07/06/26
19	19	8	2	90	80 07/06/26
20	20	8	4	180	100 07/06/26
21	21	9	1	800	1000 07/06/26
22	22	9	2	250	200 07/06/26
23	23	9	6	20	50 07/06/26
24	24	9	9	430	250 07/06/26
25	25	9	10	2	2 07/06/26
26	26	11	1	80	300 07/06/26
27	27	11	2	15	150 07/06/26
28	28	11	3	10	100 07/06/26
29	29	11	4	200	150 07/06/26
30	30	11	5	5	50 07/06/26
31	31	11	6	80	100 07/06/26
32	46	21	1	50	200 09/06/26
33	47	21	2	20	100 09/06/26
34	48	21	3	80	100 09/06/26
35	49	21	4	150	100 09/06/26
36	50	21	5	10	50 09/06/26

Figura 9 – Consulta da tabela TB_ESTOQUE_ABRIGO.

14. SEGURANÇA

As informações sensíveis da aplicação foram armazenadas utilizando variáveis de ambiente.

Nenhuma credencial foi mantida diretamente no código-fonte ou nos arquivos de configuração versionados.

15. RESULTADOS OBTIDOS

Com a implementação realizada foi possível:

- Automatizar o processo de Build;
- Automatizar o processo de Deploy;

- *Integrar Azure Boards, Repos e Pipelines;*
- *Publicar a aplicação em ambiente Azure;*
- *Persistir dados em Oracle Database;*
- *Disponibilizar uma API REST para acesso externo;*
- *Garantir rastreabilidade das atividades de desenvolvimento.*

16. CONCLUSÃO

A implementação da solução Vesta permitiu aplicar os principais conceitos relacionados a DevOps, integração contínua, entrega contínua e computação em nuvem.

A integração entre Azure Boards, Azure Repos e Azure Pipelines proporcionou maior controle sobre o ciclo de desenvolvimento da aplicação, facilitando a rastreabilidade das alterações realizadas e reduzindo as atividades manuais durante o processo de publicação.

Os objetivos definidos para o projeto foram atingidos, demonstrando a utilização prática das ferramentas estudadas ao longo da disciplina.

17. LINKS DA ENTREGA

Organização Azure DevOps

<https://dev.azure.com/kgb-easypark>

Projeto Azure DevOps

https://dev.azure.com/kgb-easypark/GS_vesta-project

Repositório GitHub

<https://github.com/Vi-debu/vesta-devops.git>

Vídeo da Demonstração

https://youtu.be/_k_xwbLtbdc